Lab 4 Practice Tasks: Process Creation

Task 1: Declare a global integer array of 5 elements, filled with the values 1 to 5. Write a program in which the parent creates one child. The child replaces every element with its square and prints the array together with its own PID and its parent's PID. The parent waits for the child, then prints its own copy of the array together with its PID and the PID of its child. Explain from the output why the two arrays differ.

Task 2: Write a program that calls fork() three times one after the other, with no conditions. After the calls, every process prints its PID and PPID. Run it and count the lines printed, then draw the process tree from your output and state how many processes exist in total and why. Finally, modify the program so that only the original parent creates the three children and the children create nothing further, and compare the number of lines printed.

Task 3: Write a program that takes an integer n from the command line. The parent creates exactly n children using a loop. Child number i prints "Child i, PID = ..., PPID = ..." and exits. The parent waits for every child and then prints how many children it reaped. Test with n = 1, 4 and 0, and with a negative n.

Task 4: Write a program that builds this tree: the original process (P) creates two children, C1 and C2, and C1 creates one child of its own, G1. Every process prints a line giving its role (P, C1, C2 or G1), its PID and its PPID. Arrange the waits so that G1 always finishes before C1, and both C1 and C2 finish before P.

Task 5: Write a program that takes an integer n from the command line and creates three children. Child 1 tests whether n is a prime number and exits with 1 if it is prime and 0 if it is not. Child 2 calculates the sum of the numbers from 1 to n and returns the sum through its exit status. Child 3 terminates abnormally because of a runtime error that you cause on purpose. The parent waits for every child and, for each one, prints the PID and says whether it exited normally (with its exit code) or was killed by a signal. Test Child 2 with n = 5 and n = 30, and explain what happens to the sum when it is larger than 255.

Task 6: Write a program in which the parent creates four children and each one runs a different program using a different member of the exec family:

- Child 1 prints the message "Hello from exec" using execl().
- Child 2 counts the number of lines in the file /etc/passwd using execv().
- Child 3 lists the files of the current directory in long format using execvp() or execlp().
- Child 4 displays the environment using the env program, with a variable of your own choice added to the environment, using execle() or execve().

The parent waits for all four and prints the PID and exit status of each. Then change one of the program names to something that does not exist and show how your error handling

reports it. In a comment, write which letters of the exec name (l, v, p, e) each of your four calls uses.

Task 7: Write a program that takes a text file name as a command line argument. Each line of the file holds one command with its arguments (for example, ls -l /tmp). The program reads the file line by line and runs every command in its own child process, one after the other, using one of the exec functions. After each command finishes, the parent prints the command, its exit status, and a message “FAILED” if the status is not zero. A command that

Task 8: Write a program in which the parent creates a child and exits after 2 seconds. The child prints its PPID once every second for 6 seconds. From the output, identify the second at which the PPID changes. Then find out the name of the process that became the new parent (use ps with the new PPID). The new parent may not be PID 1 on your system. Write, in a comment in your code, what you found and why this happens.

Task 9: Write a C program that uses the fork() system call to create a process tree in the shape of a binary tree with 7 processes, as shown in the given diagram.

Your program must satisfy the following requirements:

1. The original process P0 must create two child processes, P1 and P2.
2. Process P1 must create two child processes, P3and P4.
3. Process P2 must create two child processes, P5 and P6.
4. Each process must print its process ID (PID) and parent process ID (PPID).
5. Use the fork() system call to establish the required parent-child relationships.
6. Use wait() where appropriate to ensure parent processes wait for their children to finish.

